YEAR 1 · TERM 1 · MODULE 3 · WEEK 1

Introduction to Software Engineering Management

Software is built by teams, changes constantly, and must be traceable. This week you learn how development is organised as a process, how configuration management keeps the product under control, and how Git and GitHub let people work on the same code without losing each other's changes.

Study Guide | 8 lessons | 600 min

Contents

1. What software engineering is
2. The development lifecycle and process models
3. Requirements engineering: from wishes to specifications
4. Modelling software: from context diagrams to UML
5. Software configuration management
6. Version control systems
7. Git fundamentals: areas, commits, and branches
8. GitHub, remotes, and collaboration workflows
9. Worked solutions
10. Glossary
11. Practice quiz

Learning objectives

- Explain the difference between programming and software engineering.
- Describe a software development lifecycle and two process models.
- Write checkable requirements and tell functional from non-functional ones.
- Read and sketch basic models: context diagram, ER model, state machine.
- Define software configuration management and its main activities.
- Use basic Git commands to initialise, stage, commit, branch, and merge.
- Connect a local repository to a remote and describe a pull request workflow.

01 Lesson 1 · 15 min

What software engineering is

Programming is the act of writing instructions a computer can run. Software engineering adds the discipline needed to deliver a system that works, can be changed, and is understood by the people who depend on it. The difference matters because most software failures are not coding typos. They come from unclear requirements, poor planning, missing documentation, and communication gaps between the customer and the builders.

A useful way to see the gap is a chain of translations. The customer has a real need. They describe it imperfectly. Sales promises something. Analysts write requirements. Designers propose a structure. Programmers write code. Each handoff can lose meaning. Software engineering tries to make each handoff explicit and checkable, with written specifications, reviews, and tests.

Software is also special as a product. It is intangible, it changes often, and it is rarely finished. Engineers therefore work with models: simplified descriptions of a system, such as a diagram of the data flow or a list of use cases. A good model is abstract enough to be simple, precise enough to be trusted, and cheap enough to change before code is written.

Think of it like this

Building a house needs blueprints before bricks. Software needs plans and models before code, because changing a wall on paper costs far less than tearing it down.

Key takeaways

- Software engineering is a discipline about delivering and maintaining software, not only writing it.
- Most project problems come from communication and planning gaps.
- Models let teams discuss and check a system before building it.

Common mistakes

- ✗ Treating "it compiles" as "it meets the need".
  ✓ **Fix:** Compare the result with written requirements and tests.

Exercises

1.1 Spot the gap Level 1

A client says "make the report faster". List two questions you would ask before writing any code.

Hint: Ask what "faster" means for them: seconds, minutes, or a different data set?

1.2 Write a testable requirement Level 2

Rewrite "the login should be secure" as a requirement with a measurable condition.

Hint: Name the condition, the actor, and the expected result.

02 Lesson 2 · 20 min

The development lifecycle and process models

A software development process defines what is done, by whom, when, and how. Every process performs four broad activities: specifying the software, developing it, validating it, and evolving it after release. The lifecycle is the sequence of phases a product moves through: requirements analysis, design, implementation, testing, deployment, and maintenance.

A waterfall process runs these phases once, in order. It is easy to plan but slow to react to change, and problems discovered in testing are expensive to fix. An iterative and incremental process divides the project into iterations. Each iteration plans, analyses, designs, builds, and tests a small part, and produces a baseline: a working, partially complete version of the system. The difference between two baselines is an increment.

Unified-style processes organise iterations into phases: inception (goals and scope), elaboration (architecture and main risks), construction (building the system), and transition (delivery). Adaptive methods such as Scrum apply the same idea with short cycles and frequent feedback from users. In each case, the key question is how early the team learns something that would be expensive to learn late.

Think of it like this

Think of a cooking class. Waterfall is following a recipe from start to finish without tasting. Iterative work is tasting the sauce after each step and adjusting the next one.

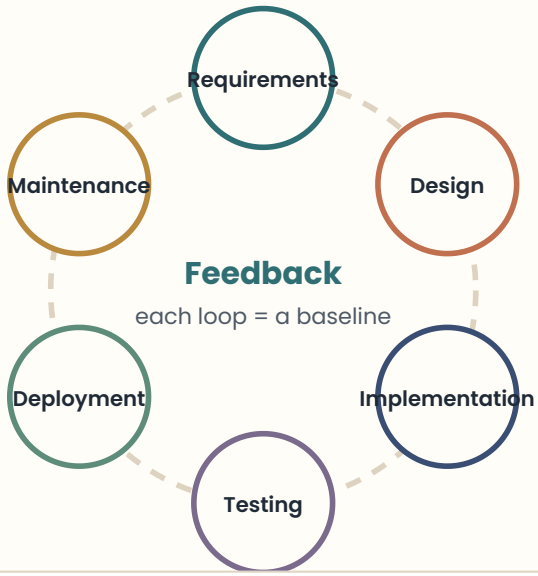


Diagram · The development lifecycle and process models

Key takeaways

- A process model is a simplified description of how real teams work.
- Iterative work produces baselines; the difference between baselines is an increment.
- Choose a process by how much change you expect and how costly late mistakes are.

Common mistakes

- ✗ Calling a waterfall project "agile" because it has meetings.
  ✓ **Fix:** Ask whether feedback changes the plan during the project.

Exercises

2.1 Pick a model Level 1

A government form has fixed legal rules that will not change for two years. Which process model fits best, and why?

Hint: Think about how much change you expect after the requirements are signed.

2.2 Draw your own baseline history Level 2

A team builds a chat app in three iterations: login, messages, and notifications. Name the baseline and increment after iteration 2.

Hint: A baseline is a working version; the increment is the difference from the previous one.

Lesson 3 · 25 min

03 Requirements engineering: from wishes to specifications

A requirement is a capability or condition a system must have to be accepted by its users, its customer, or a standard. Requirements engineering turns a business wish into precise, checkable statements that analysts, designers, programmers, and testers all share. Mistakes here are the most expensive kind, because every later phase is built on top of them.

The work flows through seven activities that often overlap. Inception identifies the need and a high-level solution. Elicitation gathers needs from stakeholders. Elaboration refines them into scenarios, business rules, and main and alternative flows. Negotiation reconciles conflicting views and weighs risk, cost, and time. Specification writes everything down in a standard template, often with diagrams and prototypes. Validation reviews the specification with users and fixes problems before building. Management tracks every change to the requirements for the whole project.

Elicitation is hard for three reasons. Scope problems: the system boundaries are unclear, so the analyst writes assumptions and constraints that say what is in and what is out. Understanding problems: users may not know what they need, or describe it at too high a level. Volatility: requirements change from one meeting to the next, so teams record agreements in written minutes and work with one representative user.

Requirements come in layers. Business requirements state the organisation's goals. Stakeholder requirements describe the needs of a group of users. Solution requirements say what the system must do, and split into functional requirements (behaviour: "the system emails a receipt after payment") and non-functional requirements (qualities: "a page loads in under 2 seconds with 500 users online"). Transition requirements are temporary capabilities to move from the old situation to the new one, such as migrating old data.

Think of it like this

A tailor measures twice and cuts once. Requirements are the measurements: if they are wrong, no amount of careful sewing (coding) will make the suit fit.

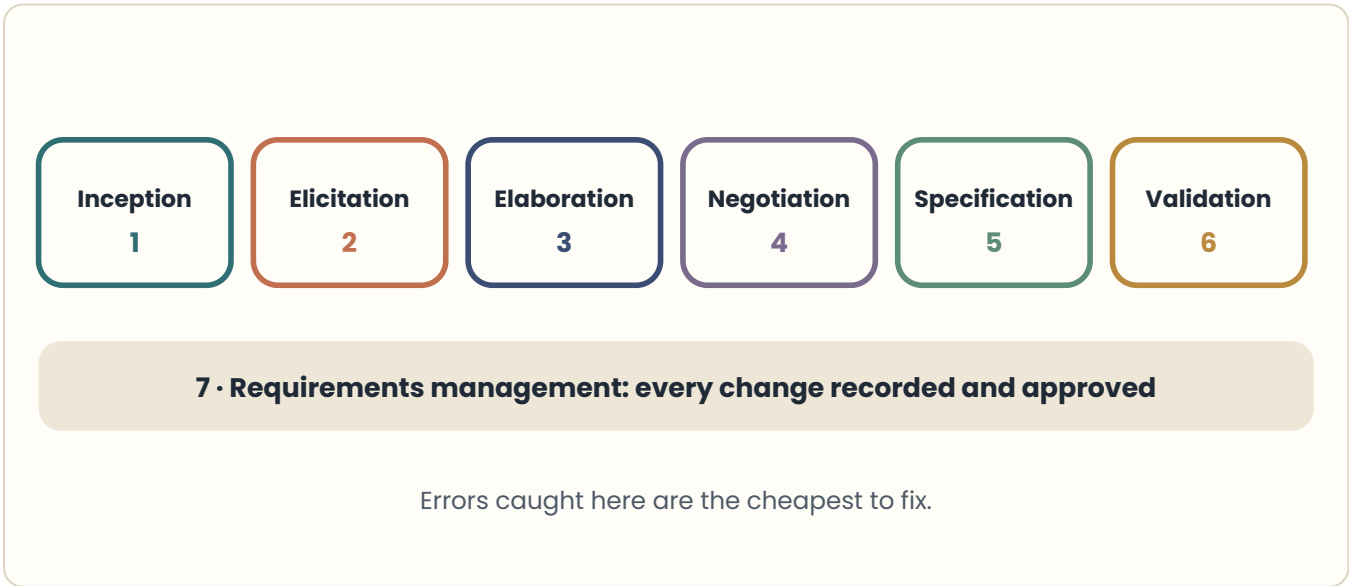


Diagram · Requirements engineering: from wishes to specifications

Key takeaways

- A requirement must be checkable: someone should be able to say "met" or "not met".
- Seven activities: inception, elicitation, elaboration, negotiation, specification, validation, management.
- Functional = what the system does. Non-functional = how well it does it.

Common mistakes

- ✗ Writing "the system should be fast and easy".
  ✓ **Fix:** Add a measure: "search results appear in under 1 second".
- ✗ Changing requirements by chat with no record.
  ✓ **Fix:** Record each agreement and its approver; that is requirements management.

Exercises

3.1 Find the boundary Level 1

A library asks for "a system for books". Write two assumptions or constraints that set the scope.

Hint: Say what is included and what is explicitly out.

3.2 Split the layers Level 2

For an online pharmacy, write one business, one functional, one non-functional, and one transition requirement.

Hint: Business = goal; functional = action; non-functional = quality with a number; transition = temporary.

Lesson 4 · 25 min

04 Modelling software: from context diagrams to UML

A model is a simplified representation of a system that helps you reason about it. Good models have five qualities: they are abstract (they leave out what does not matter), understandable, precise (faithful to the system), predictive (they support correct conclusions), and cheaper than building and studying the real thing. Models become the shared language between users, analysts, and developers.

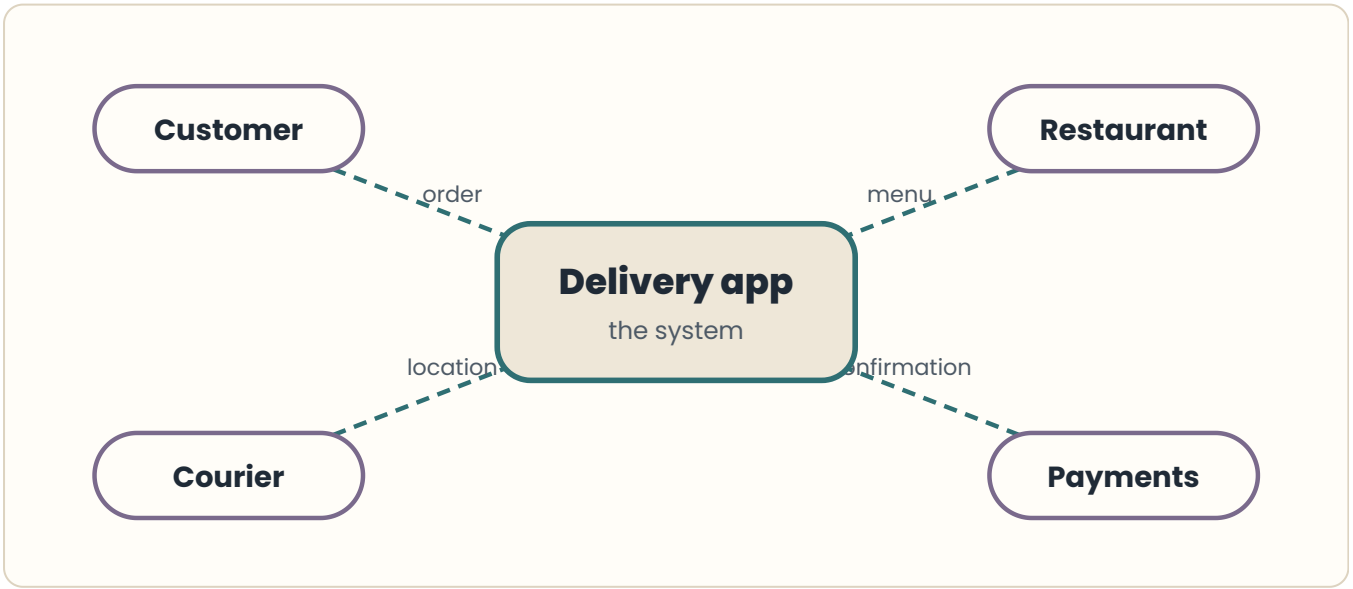
Analysis models describe the problem in the real world: what the business does. Design models describe the software solution: how the system will do it. Forward engineering goes from a model to working code; reverse engineering rebuilds a model from an existing product or code base, which is common when documentation is missing.

Early in a project, a context diagram draws the system as a single box surrounded by the external actors and the data that flows in and out; it fixes the boundaries. A data flow diagram then breaks the process into steps with inputs and outputs. For data, an entity-relationship model shows entities, their attributes, and relationships with cardinality: one-to-one, one-to-many, or many-to-many. In a table, a primary key identifies each row, and a foreign key stores another table's primary key to link them. A state machine shows the states a component can be in, such as Draft, Paid, and Shipped, and the events that move it between them. A data dictionary defines every data item, so everyone uses the same meaning.

UML, the Unified Modeling Language, standardises diagrams for object-oriented development. Structure diagrams show the parts of a system: class, component, object, package, and deployment diagrams. Behaviour and interaction diagrams show how it acts: use case, activity, state, and sequence diagrams. You rarely need all of them. Pick the diagram that answers the question your team is asking right now.

Think of it like this

A city map is a model. It ignores the colour of each house (abstract) but gets the streets right (precise), so you can plan a route before walking (predictive and cheap).



Key takeaways

- A model is useful when it is simpler than the system but still correct about what matters.
- Context diagram = boundaries; ER model = data; state machine = lifecycle of one thing.
- UML has structure diagrams and behaviour diagrams; choose by the question you need answered.

Common mistakes

- ✗ Drawing every UML diagram "to be complete".
  ✓ **Fix:** Draw the one diagram that answers the current question.

Exercises

4.1 Draw a context diagram in words Level 1

For a food delivery app, list the external actors and one data flow in and out for each.

Hint: Think of customers, restaurants, couriers, and the payment provider.

4.2 Design a state machine Level 3

List the states of an online order and the event for each transition.

Hint: Start at Created; end at Delivered or Cancelled.

05

Lesson 5 · 15 min

Software configuration management

Software configuration management (SCM) controls how the parts of a system change over time. Its items are not only source code: requirements documents, designs, test scripts, build files, and manuals are all configuration items. The goal is to know exactly which version of each item belongs to which version of the product.

SCM rests on four ideas. Identification names each item and version. Version control records every change. Change control decides which changes are approved and when. Status accounting reports what is currently in each baseline. An audit then checks that the baseline matches what was intended.

Without SCM, teams struggle with questions like "which build is in production?" or "why did this test pass yesterday?". Keeping a stable baseline lets people keep working in parallel without overwriting each other.

Think of it like this

A library keeps a catalogue of every edition and who borrowed it. SCM does the same for a product: every item, version, and change has a record you can check.

Key takeaways

- Configuration items include everything that makes up the product, not only code.
- SCM answers: which version, which change, approved by whom, and in which baseline.

Exercises

5.1 Inventory the configuration items Level 1

List five configuration items for a mobile app that are not source code.

Hint: Think about documents, tests, build settings, and releases.

5.2 Trace a bug to a baseline Level 3

Production shows a bug that was not in the test build. Describe the four SCM questions you would answer to find the cause.

Hint: Identification, version control, change control, status accounting.

06 Lesson 6 · 20 min

Version control systems

A version control system (VCS) records changes to files over time. It gives you a history of what changed, when, and by whom; the ability to recover earlier versions; and a way for several people to work on the same project without losing each other's work.

Centralised systems keep one master repository on a server. Distributed systems, such as Git, give every developer a full local copy of the repository and its history. You can commit, branch, and browse history offline, and share changes when you are connected. Git is a distributed VCS. GitHub, GitLab, and similar services host Git repositories and add collaboration features; they are not Git itself.

A commit is a snapshot of the project with a message. Good commits represent one logical change, and their messages say what changed in the imperative mood, for example "add sum functionality" or "fix off-by-one in pagination". Small, frequent commits make history easier to read and to revert.

Think of it like this

Version control is like a video game with save points. You can go back to any save, see what changed between two saves, and let friends play from their own copy.

Key takeaways

- A VCS stores history, supports recovery, and shows who changed what and when.
- Git is distributed; GitHub is a hosting service for Git repositories.
- One commit should hold one logical change with a clear message.

Common mistakes

- ✗ Writing commit messages like "stuff" or "changes".
  ✓ **Fix:** Describe the change in an imperative line of about 50 characters.

Exercises

6.1 Fix the commit message Level 1

Rewrite these messages in the imperative style: "stuff", "Fixed the bug.", "Added a new function for sums".

Hint: Start with a verb, lower-case, no final full stop, about 50 characters.

6.2 Split a commit Level 2

One commit mixes a bug fix, a rename, and a new test. Describe how you would split it into three logical commits.

Hint: Stage related files together and commit each group in turn.

07 Lesson 7 · 30 min

Git fundamentals: areas, commits, and branches

Git has three areas. The working directory holds the files you edit. The staging area (also called the index) holds the changes you have chosen for the next commit. The repository, stored in the hidden .git folder, holds the committed history. Most commands move changes between these areas: git add moves work into staging, and git commit moves staged work into the repository.

The everyday loop is: edit, check, stage, commit. Run git status often; it shows which files are modified, which are staged, and which are untracked. Use git log --oneline --graph to read history. Use git diff to see unstaged changes and git diff --staged to see what the next commit will contain.

A branch is a movable pointer to a commit. Creating a branch does not copy files. It lets you develop a feature without changing main. When the feature is ready, you merge it back. If both branches changed the same lines, Git stops and marks a conflict: the file contains markers <<<<<<<, =======, and >>>>>>> showing both versions. You edit the file to the intended result, stage it, and commit.

Think of it like this

Git is a photo studio. The working directory is the set, staging is choosing which photos go in the album, and commit is printing the album page. A branch is a second album you can build without touching the first.

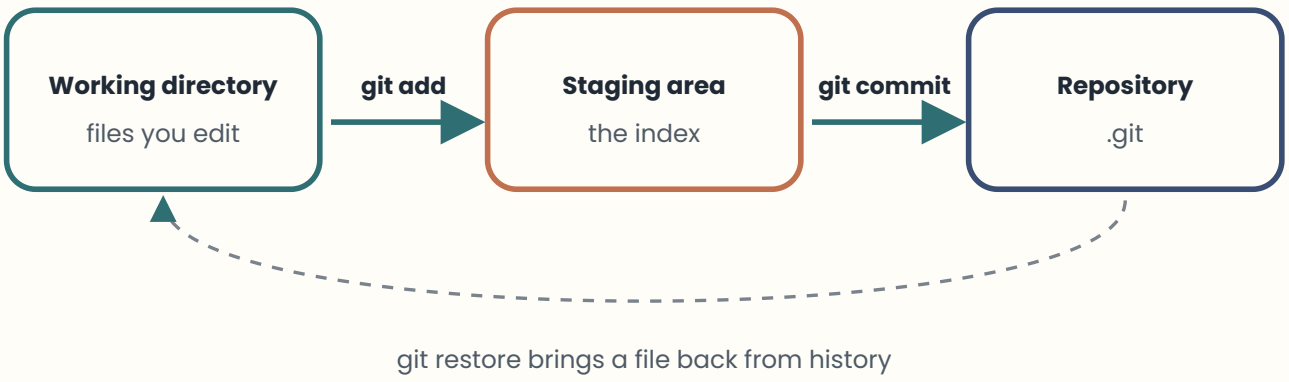


Diagram · Git fundamentals: areas, commits, and branches

Key takeaways

- Edit in the working directory, stage with git add, and record with git commit.
- git status and git log show where you are; use them before and after each step.
- A branch is a pointer. Merging combines history; conflicts require a human decision.

Common mistakes

- ✗ Committing without checking git status, so unrelated files get included.
  ✓ **Fix:** Run git status and git diff --staged before each commit.
- ✗ Deleting conflict markers without choosing the right content.
  ✓ **Fix:** Decide the final content for the lines, remove all three marker lines, then stage and commit.

Exercises

7.1 Predict the status Level 1

You edit a.txt, then run git add a.txt, then edit a.txt again. Is the second edit staged? What does git status show?

Hint: Staging captures the file at the moment you run git add.

7.2 Read the graph Level 2

Run git log --oneline --graph --all in a sandbox after Lab 2. Explain in one sentence why main and feature-sum point to the same commit.

Hint: What happens when the branch you merge into has no new commits?

7.3 Resolve by intent Level 3

Two branches changed the same function: one renamed a parameter, the other added validation. Describe the resolution you want and the commands.

Hint: Keep both intents, then stage and commit.

08 Lesson 8 · 25 min

GitHub, remotes, and collaboration workflows

A remote is a named link to another repository, conventionally called origin. git remote add origin <url> registers it. git push -u origin main uploads your main branch and sets the upstream. git fetch downloads new commits without changing your files; git pull fetches and then integrates them into your current branch.

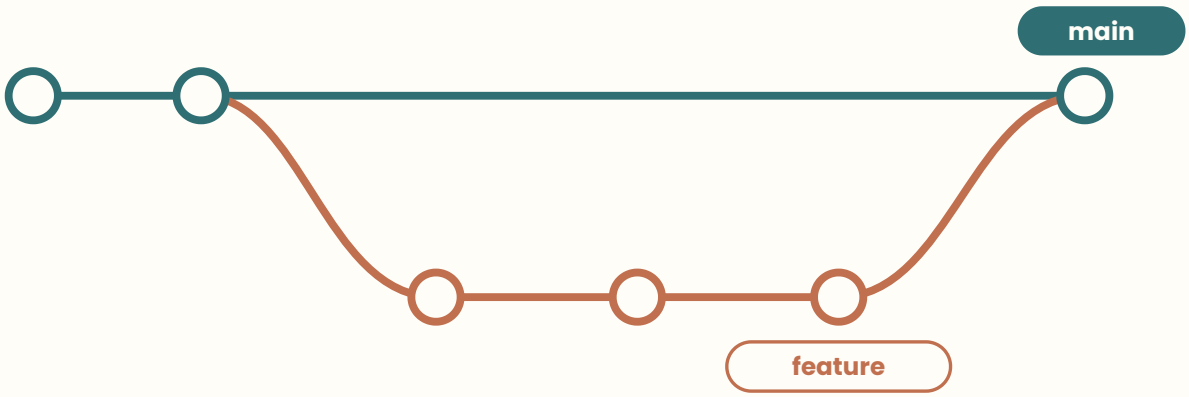
A common team workflow is feature branches with pull requests (called merge requests on GitLab). You create a branch, push it, and open a request. Reviewers read the diff, comment, and approve. Only after approval is the branch merged into main. The review step is where teams catch defects and share knowledge.

Good collaboration also means keeping main stable, updating your local copy before you start work, and writing commits that a reviewer can understand one at a time.

Think of it like this

A pull request is like a draft essay sent to a classmate before handing it in. The reviewer does not write it for you, but catches mistakes while they are cheap to fix.

Branch, commit, merge back



Each circle is a commit. A branch is a pointer. Merging joins histories.

Diagram · GitHub, remotes, and collaboration workflows

Key takeaways

- origin is the conventional name for the remote you cloned or connected.
- fetch downloads; pull downloads and integrates; push uploads.
- Pull requests make review a planned step before code reaches main.

Common mistakes

- ✗ Pushing directly to main on a shared project.
  ✓ **Fix:** Work on a feature branch and open a pull request for review.

Exercises

8.1 Order the workflow Level 1

Put these in order: merge, open pull request, push feature branch, review and approve, create branch.

Hint: You cannot open a request before the branch exists on the remote.

8.2 Recover from a rejected push Level 3

A teammate pushed to main while you were working. Your git push is rejected. Write the safe sequence of commands.

Hint: Fetch first, integrate, then push. Never force.

Worked solutions

1.1 Spot the gap. Example: "Which report, and how slow is it now versus the target?" and "Who uses it, and how often?" Each answer turns a vague wish into a requirement you can test.

1.2 Write a testable requirement. Example: "After 5 failed logins within 10 minutes, the account is locked for 15 minutes and the user receives an email." A tester can now check it.

2.1 Pick a model. A sequential (waterfall-style) process can fit, because requirements are stable and early mistakes would be expensive. An iterative process would also work if the team wants early feedback on the interface.

2.2 Draw your own baseline history. Baseline 2 is login plus messages, working and documented. The increment is the messaging feature compared with baseline 1.

3.1 Find the boundary. Example: "Includes loans and returns for members." "Excludes buying new books and paying staff salaries." Clear limits prevent scope problems.

3.2 Split the layers. Business: "increase online sales by 20% this year." Functional: "customers upload a prescription photo." Non-functional: "orders are confirmed within 5 seconds." Transition: "import 10 years of customer records from the old system."

4.1 Draw a context diagram in words. Customer: order in, status out. Restaurant: menu in, order out. Courier: location in, pickup task out. Payment provider: confirmation in, charge request out.

4.2 Design a state machine. Created --pay--> Paid --pack--> Packed --ship--> Shipped --deliver--> Delivered. Created or Paid --cancel--> Cancelled. Each arrow is an event; each box is a state.

5.1 Inventory the configuration items. Examples: requirements specification, UI design files, test plan and test scripts, build configuration files, release notes, user manual.

5.2 Trace a bug to a baseline. Which exact versions are in production (identification)? What changed between the test build and production (version control)? Which changes were approved and by whom (change control)? Is the production build the one recorded in the baseline (status accounting)?

6.1 Fix the commit message. fix login redirect loop | fix bug where … (name the bug) | add sum function. Specific verbs help reviewers scan history quickly.

6.2 Split a commit. Stage the bug fix files, commit; stage the renamed files, commit; stage the test, commit. Each commit can now be reviewed or reverted alone.

7.1 Predict the status. No: the second edit is not staged. git status lists a.txt under both "Changes to be committed" (the first version) and "Changes not staged for commit" (the newer edit).

7.2 Read the graph. main had no new commits since feature-sum split, so Git moved the pointer forward (fast-forward) instead of creating a merge commit.

7.3 Resolve by intent. Edit the function so it uses the new parameter name and includes the validation. Remove all markers, run git add on the file, then git commit to finish the merge.

8.1 Order the workflow. Create branch → push feature branch → open pull request → review and approve → merge.

8.2 Recover from a rejected push. git fetch origin, then git pull origin main (or git rebase origin/main), resolve any conflicts, run tests, then git push origin main.

Glossary

Question	Answer
What is the main difference between programming and software engineering?	Programming writes code. Software engineering also plans, specifies, validates, and maintains a system that meets a real need.
Why do engineers build models before writing code?	Models are cheaper to change than code. They simplify the system, make it easier to discuss, and let you predict behaviour.
What does a software development process define?	What is done, by whom, when, and how. Its four common activities are specification, development, validation, and evolution.
Waterfall vs iterative and incremental: what is the key difference?	Waterfall runs each phase once, in order. Iterative work repeats the phases in short cycles and delivers working increments.
What is a baseline?	An agreed, stored version of the system and its documents. Later work is measured against it.
What is an increment?	The difference between two successive baselines.
Name three kinds of configuration item besides source code.	Any three of: requirements documents, designs, test scripts, build files, manuals.
Change control vs status accounting	Change control decides which changes are approved. Status accounting reports what is currently in each baseline.
Give two reasons to use a version control system.	Keeps history, recovers earlier versions, shows who changed what and when, supports parallel work.
Centralised vs distributed version control	Centralised: one master repository on a server. Distributed: each developer has a full local copy with history, as in Git.
Git vs GitHub	Git is the version control tool. GitHub is a hosting service for Git repositories with review and collaboration features.
What is a good commit?	One logical change, small and frequent, with an imperative message such as "add sum functionality".
Name the three areas of Git.	Working directory, staging area (index), and repository (.git).
What does git add do?	Moves changes from the working directory into the staging area. It does not create a commit.
What does git commit -m "message" do?	Records the staged changes as a new commit with the message.
Why run git status before committing?	It shows modified, staged, and untracked files, so you do not commit unrelated changes.
What does git log --oneline --graph show?	A compact commit history with short hashes, messages, and branch lines.
What is a branch in Git?	A movable pointer to a commit that lets you develop work in isolation. Creating one copies no files.

What causes a merge conflict?	Two branches changed the same lines differently. Git cannot decide, so a person must choose the result.
What is origin?	The conventional name for a remote repository, such as the GitHub copy you cloned from.
git fetch vs git pull	fetch downloads new remote commits without changing your files. pull fetches and then integrates them into your current branch.
What does git push -u origin main do?	Uploads your local main branch to origin and sets the upstream, so later push and pull need no arguments.
Why use pull requests?	They make review a planned step: reviewers read the diff, comment, and approve before the branch merges into main.
Why is pushing directly to a shared main risky?	Untested or unreviewed changes reach everyone at once, and history on shared branches should not be rewritten.
What makes a statement a good requirement?	It is a capability or condition the system must meet, and someone can check whether it is met.
Name the seven requirements engineering activities.	Inception, elicitation, elaboration, negotiation, specification, validation, management.
Three reasons elicitation is hard?	Scope (unclear boundaries), understanding (users cannot say what they need), volatility (needs keep changing).
Functional vs non-functional requirement	Functional: a behaviour ("send a receipt"). Non-functional: a quality with a measure ("under 2 seconds").
What is a transition requirement?	A temporary capability to move from the old state to the new one, such as migrating old data.
Five qualities of a good model?	Abstract, understandable, precise, predictive, cheap.
Analysis model vs design model	Analysis describes the real-world problem. Design describes the software solution.
What does a context diagram show?	The system as one box, the external actors around it, and the data flowing in and out. It fixes the boundaries.
Primary key vs foreign key	A primary key identifies each row of a table. A foreign key stores another table's primary key to link the two.
UML structure vs behaviour diagrams	Structure: class, component, package, deployment. Behaviour: use case, activity, state, sequence.

Practice quiz

1. Which item is a configuration item in software configuration management?

A. A requirements document
B. The office coffee machine
C. A personal calendar

2. Which statement about Git is correct?

A. Git is a distributed version control system; every clone holds the full history.
B. Git cannot record a commit without an internet connection to GitHub.
C. Git and GitHub are two names for the same tool.

3. In an iterative and incremental process, what does the difference between two successive baselines represent?

A. An increment
B. A risk register
C. The final delivery only

4. True or false: Git and GitHub are the same product.

A. True
B. False

5. True or false: git add creates a commit in the repository history.

A. True
B. False

6. True or false: creating a Git branch copies every file in the project.

A. True
B. False

7. Changes chosen for the next commit wait in the _____ area. (Hint: also called the index.)

Answer: _____

8. Type a command that creates and switches to a branch named feature-login (modern syntax).

Answer: _____

9. In two sentences, explain the difference between git fetch and git pull.

Answer: _____

10. Give one benefit of small, focused commits.

Answer: _____

11. Match each command with its purpose.

Match each item on the left with one on the right.

1. git add	A. Shows modified, staged, and untracked files
2. git commit	B. Moves changes into the staging area
3. git status	C. Downloads remote commits without merging
4. git fetch	D. Records staged changes as a new commit

12. Match each process model with its main characteristic.

Match each item on the left with one on the right.

1. Waterfall	A. Short cycles with frequent feedback from users
2. Iterative and incremental	B. Phases run once, in order
3. Adaptive (for example, Scrum)	C. Repeated iterations that produce baselines

---

13. Interpret the commands. What does git status report for notes.txt?

```
echo "v2" >> notes.txt
git add notes.txt
git status
```

A. notes.txt is listed under "Changes to be committed"
B. notes.txt is already in the repository history
C. notes.txt is unchanged

14. Interpret the command. What will the output show?

```
git log --oneline -3
```

A. The three most recent commits, one per line, with short hashes and messages
B. The three oldest files in the project
C. The three branches with the most conflicts

15. A teammate says the last release behaves incorrectly. Which question should you ask first?

A. Which configuration items and versions are in that baseline?
B. Who has the fastest laptop?
C. Can we delete the repository and start again?

16. You want a teammate to check your login change before it reaches main. What do you do?

A. Push a feature branch and open a pull request for review
B. Push directly to main and ask for feedback afterwards
C. Email a zip file of the project

17. Two branches changed the same line in app.js. The merge stops. What happened?

A. A merge conflict; a person must choose the final content
B. Git deleted the repository
C. The remote is offline

18. git push is rejected because the remote has commits you do not have locally. What is the safest fix?

A. Run git fetch, integrate the remote changes (merge or rebase), then push again
B. Run git push --force to overwrite the remote
C. Delete your local repository and clone again

19. git commit fails with "Author identity unknown". What should you do?

A. Set your name and email with git config --global user.name and user.email
B. Delete the .git folder
C. Commit with git add -f

20. After a merge, a file still contains <<<<<<< HEAD lines. What is the correct resolution?

A. Edit the file to the intended content, remove the marker lines, git add the file, then commit
B. Leave the markers and commit anyway
C. Run git merge --abort after committing

21. Which statement is a non-functional requirement?

A. Search results appear in under one second
B. Users can reset their password by email
C. Admins can export a sales report

22. True or false: transition requirements stay in the system permanently.

A. True
B. False

23. Every meeting, a different user changes what was agreed last week. Which elicitation problem is this, and what helps?

A. Volatility: record agreements in minutes and work with one representative user
B. Scope: buy a faster server
C. Understanding: start coding immediately

24. Match each activity with what it does.

Match each item on the left with one on the right.

1. Elicitation	A. Reviews the specification with users before building
2. Negotiation	B. Gathers needs from stakeholders
3. Validation	C. Tracks every change to requirements
4. Management	D. Reconciles conflicting views, cost, and risk

---

25. Which model best shows the boundaries of a system and who exchanges data with it?

A. Context diagram
B. State machine
C. Class diagram

26. A column that stores another table's primary key to link the two tables is called a ____ key.

Answer: _____

Answer key

1. A — Any artefact that is part of the product or its documentation can be a configuration item. Requirements are a core example.
2. A — Each clone has the full repository and history, so you can commit offline.
3. A — By definition, the difference between two baselines is an increment.
4. B — False. Git is the version control tool. GitHub hosts Git repositories and adds collaboration features.
5. B — False. git add only stages changes. git commit records them.
6. B — False. A branch is a pointer to a commit. Git does not copy the files when you create one.
7. The staging area, also called the index, holds the changes that will go into the next commit.
8. git switch -c creates and switches. git checkout -b does the same with older syntax.
9. fetch downloads remote commits without changing your working branch. pull fetches and then integrates them into your current branch.
10. Small commits are easier to review, understand, and revert if a single change breaks something.
11. 1→B, 2→D, 3→A, 4→C — add stages, commit records, status reports, and fetch downloads remote commits without changing your files.
12. 1→B, 2→C, 3→A — Waterfall is sequential; iterative work produces baselines; adaptive methods run short feedback cycles.
13. A — The edit is staged by git add, so git status lists it as ready to commit. Nothing is in history until git commit runs.
14. A — --oneline prints one commit per line, and -3 limits the output to three commits, newest first.
15. A — Before debugging, identify exactly which versions were released. SCM makes this traceable.
16. A — A feature branch with a pull request lets reviewers see the diff and approve before merging.
17. A — Git cannot choose between two different edits to the same lines, so it marks the conflict for you to resolve.
18. A — Integrate the remote work first. A forced push can erase a teammate's commits and should be avoided on shared branches.
19. A — Git stores the author name and email in every commit, so both must be configured first.
20. A — Markers are not valid code. Choose the final content, remove all markers, stage the file, and complete the merge with a commit.
21. A — It describes a quality (speed) with a measure. The others describe behaviours, so they are functional.
22. B — False. They are temporary, needed only to move from the old situation to the new one.
23. A — Requirements that keep changing are volatile. Written agreements and a single representative make them stable.
24. 1→B, 2→D, 3→A, 4→C — Elicitation gathers, negotiation reconciles, validation reviews, and management tracks change.
25. A — A context diagram draws the system as one box with its external actors and data flows.
26. That is a foreign key. It creates relationships such as one customer with many orders.